
WHEN MARKDOWN HURTS RAG: SHALLOW STRUCTURAL CHUNKS AS DENSE RETRIEVAL FALSE POSITIVES

A PREPRINT

📧 Antonio Cioffi

Independent Researcher

github.com/cioffiAI/rag-vs-markdown

ABSTRACT

Retrieval-Augmented Generation (RAG) pipelines typically embed raw document text into a vector database. An alternative—preprocessing documents into structured Markdown before indexing—promises better chunk coherence, preserved table structure, and clearer section boundaries. We benchmark both strategies across four models (Gemma 3 4B, Nemotron 3, DeepSeek V4 Flash, Gemma 4 26B) on a 50-question corpus of 10 arXiv papers. All four models consistently prefer raw text over Markdown (−3.6% to −6.0%). We identify the mechanism: 25.3% of retrieved Markdown chunks are *shallow*—section headers and table markup with ≤ 200 informative characters—that compete with substantive chunks via cosine similarity, producing embedding false positives. A third pipeline that filters out these shallow chunks partially recovers the loss (+0.07 to +0.17). An oracle test reveals an +85% retrieval ceiling. These findings link the document preprocessing choice to a specific retrieval failure mechanism and show that the cost of structured markup can outweigh any structural benefit in our setup.

Keywords Retrieval-Augmented Generation · document preprocessing · Markdown · embedding quality · model-pipeline interaction

1 Introduction

Retrieval-Augmented Generation (RAG) [Lewis et al., 2020] has become a standard architecture for grounding large language model (LLM) outputs in external knowledge. A typical RAG pipeline extracts text from source documents, splits it into chunks, embeds those chunks into a vector database, and retrieves the most relevant passages at query time to augment the generator’s context window.

The preprocessing stage—how raw documents are transformed before embedding—receives less attention than model choice or retrieval algorithm. Most pipelines operate on raw extracted text. An alternative, which we call *knowledge compilation*, converts source PDFs into structured Markdown before indexing: section headers become heading markers, tables become pipe-separated grids, and metadata (page numbers, artifacts) are removed. The intuition is straightforward: if the embedding space better reflects document structure, retrieval quality should improve.

We test this intuition systematically. Three pipelines—Raw (PDF text extracted directly via PyMuPDF), Markdown (PDF $\rightarrow$ structured Markdown compiled by a rule-based parser), and Filtered-Markdown (Markdown with shallow chunks removed)—are evaluated on the same 50-question benchmark with the same embedding model (all-MiniLM-L6-v2) and vector database (ChromaDB, cosine distance). Four language models (Gemma 3 4B, Nemotron 3 Super Free, DeepSeek V4 Flash, Gemma 4 26B) serve as generators.

Our central finding is that Markdown preprocessing consistently underperforms raw text across all four models tested (by −3.6% to −6.0%). We identify a specific retrieval failure mode we call *shallow chunks*: structurally rich but informationally empty Markdown fragments—section headers, list markers, table formatting artifacts—that capture embedding similarity via structural keywords while contributing no factual content. A third pipeline that filters these

shallow chunks (9 out of 553 chunks, 1.6% of the collection) partially recovers the loss for all models (+0.07 to +0.17), confirming the mechanism. An oracle test that bypasses retrieval entirely reveals an +85% retrieval ceiling.

The paper makes three contributions:

1. A **4-model, 3-pipeline benchmark** consistently showing Markdown hurts retrieval quality, with shallow chunks as the identified mechanism.
2. The **formalization of shallow chunks** as a retrieval failure mode unique to structured preprocessing, with quantitative evidence (25.3% of retrieved Markdown chunks).
3. A **confirmatory intervention**—filtering shallow chunks recovers 1.4–3.4% of the score—that validates the mechanism.

2 Related Work

2.1 Document Preprocessing for RAG

The impact of chunking strategy on retrieval quality is well-studied. Gao et al. [2023] compare chunk sizes and overlap for academic question answering. Chen et al. [2024] evaluate semantic vs. fixed-size chunking. However, most work treats chunking as a downstream parameter of the embedding model, not as a *document transformation* that alters information structure before embedding. Our focus is the document representation format itself—raw vs. Markdown—which differs from chunk-size tuning in that it changes *what text sequences exist to be chunked*.

Markdown preprocessing for RAG appears in practitioner guides [OpenAI, 2024, Cursor, 2024] and data-loading libraries such as LlamaIndex [Ila, 2024], which offer PDF-to-Markdown converters. To our knowledge, no controlled experimental comparison quantifies how this transformation interacts with generator capability.

2.2 Embedding False Positives in Dense Retrieval

Dense retrieval models encode text into a continuous vector space where cosine similarity approximates semantic relevance [Karpukhin et al., 2020]. However, the embedding space can be misled by lexical overlap: Reimers and Gurevych [2019] note that sentence embeddings capture surface-form similarity even when semantic content differs. Our shallow chunk finding instantiates this: Markdown headers are lexically rich but semantically empty, and the embedding model retrieves them preferentially.

2.3 Model Scale and Instruction Following

Larger models generally perform better on RAG benchmarks [Lewis et al., 2020, Izacard et al., 2022]. However, the interaction between model scale and document format is under-explored. Liu et al. [2024] find that larger models “get lost” in long contexts, suggesting that format-induced token overhead (such as Markdown markup) may disproportionately affect models that are otherwise capable.

2.4 Error Analysis in RAG

Error taxonomies for RAG exist at both the retrieval and generation levels [Barnett et al., 2024, Es et al., 2023]. Our work adopts a 7-code taxonomy (E01–E07) that separates retrieval errors from generation errors, consistent with frameworks proposed by Saad-Falcon et al. [2023] and Yang et al. [2024]. We extend these by adding an oracle-condition test that bounds the retrieval contribution.

3 Experimental Setup

3.1 Pipelines

We compare three document preprocessing pipelines (Figure 1):

Pipeline A (Raw): PDF → PyMuPDF extraction → raw text → paragraph-preserving chunking → all-MiniLM-L6-v2 embedding → ChromaDB (cosine). Produces 542 chunks.

Pipeline B (Markdown): PDF → PyMuPDF extraction → raw text → rule-based Markdown compilation (headings, tables, lists) → paragraph-preserving chunking → all-MiniLM-L6-v2 embedding → ChromaDB (cosine). Produces 553 chunks.

Pipeline C (MD-filtered): Same as Pipeline B, but chunks with ≤ 200 informative characters (after stripping Markdown syntax) are removed before embedding. Keeps 544 of 553 chunks.

Both pipelines use the same chunker: a paragraph-preserving algorithm that accumulates whole paragraphs into a buffer and closes it when the next paragraph would exceed 1,500 characters. Single paragraphs longer than 1,500 characters are stored as standalone chunks. The nominal chunk size is therefore a closing threshold, not a hard limit; average chunk lengths are approximately 3,100 (Pipeline A) and 3,500 (Pipeline B) characters. Overlap is 150 characters to ensure continuity across chunk boundaries.

The Markdown compiler (`compile_markdown.py`) identifies section headers from font-size and position heuristics, reconstructs tables from spatial alignment, and strips PDF artifacts (page numbers, headers). It is a deterministic, rule-based transformation with no learned components.

3.2 Corpus and Questions

The corpus consists of 10 well-known arXiv papers spanning NLP, computer vision, and evaluation:

- RAG (Lewis et al., 2020), Self-RAG (Asai et al., 2023), Transformer (Vaswani et al., 2017)
- YOLOv7 (Wang et al., 2022), LLM Evaluation Survey (Chang et al., 2023)
- Foundation Models (Bommasani et al., 2021), Ragas (Es et al., 2023)
- CRAG (Yang et al., 2024), HELM (Liang et al., 2022)
- Tree-of-Thoughts (Yao et al., 2023), used as a negative control

We constructed 50 gold-standard questions in Italian (to isolate RAG performance from the LLM’s English parametric knowledge of these famous papers) across five types:

Table 1: Benchmark question types.

Type	Count	Description
Simple factual	15	Single-document, direct extraction
Local reasoning	10	Multi-paragraph, same document
Multi-document	10	Compare/contrast across 2+ papers
Table extraction	5	Numeric values from tables
Negative (trap)	10	Not answerable from corpus

Negative questions include three subtypes: *true absence* (the answer genuinely does not exist in any document), *false premise* (the question assumes incorrect facts), and *out-of-domain* (the question refers to a paper in the corpus but on an unrelated topic). Each question includes manually written key points for fuzzy scoring.

3.3 Models

We test four language models accessed via different providers:

Table 2: Models and providers. Parameter counts are approximate; DeepSeek V4 Flash is undisclosed.

Model	Params	Provider	API
Gemma 3 4B	4B	Ollama Cloud	OpenAI-compatible
Nemotron 3 Super Free	~3B	OpenCode Zen	OpenAI-compatible
DeepSeek V4 Flash	—	OpenCode Go	OpenAI-compatible
Gemma 4 26B A4B	~4B active / 26B total	Google Gemini	Gemini REST

3.4 Experimental Parameters

Confound note: These models differ along multiple correlated axes—parameter count, provider, API protocol, prompt formatting, system prompt enforcement, and instruction-following behavior. While all four models prefer Raw over Markdown, the magnitude differs (from -3.6% to -6.0%), and this variation is confounded by provider identity. We report the pattern as consistent across all tested configurations, not as a controlled comparison of model capability.

Figure 1. Pipeline overview

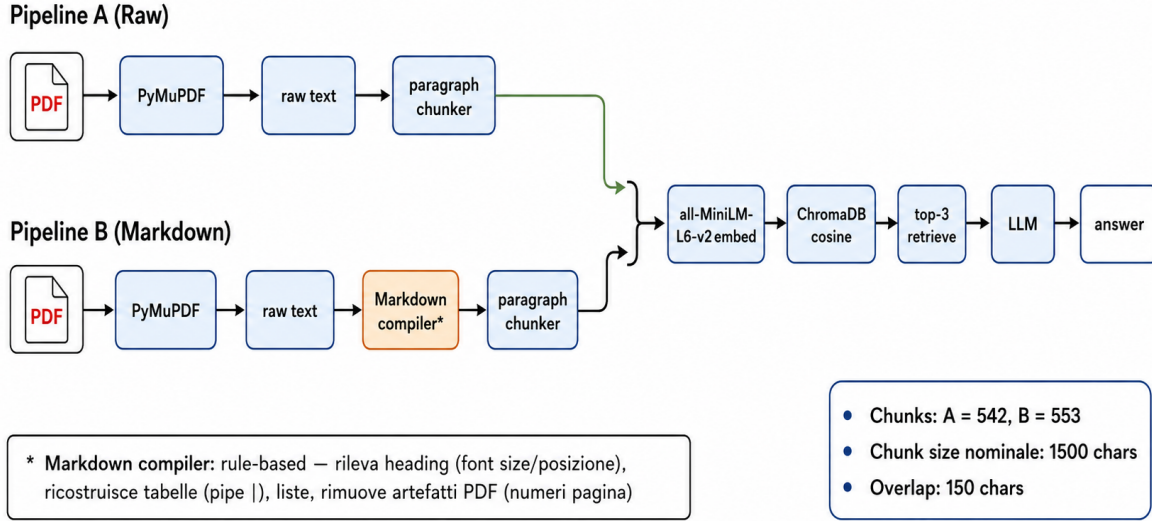


Figure 1: Simplified pipeline comparison. Pipeline A (Raw) processes PDF text directly; Pipeline B (Markdown) adds a rule-based Markdown compilation step; Pipeline C applies a shallow chunk filter after Markdown compilation. All share the same chunker, embedding model (all-MiniLM-L6-v2), vector database (ChromaDB, cosine), and LLM generator.

Table 3: Experimental parameters shared across all models and pipelines.

Parameter	Value
Embedding model	all-MiniLM-L6-v2 (sentence-transformers)
Embedding device	CPU
Vector database	ChromaDB (persistent, cosine distance)
Chunking strategy	Paragraph-preserving closing threshold
Nominal chunk size	1,500 characters
Overlap	150 characters
Min chunk length	40 characters
Retrieval depth	TOP_K = 3
LLM temperature	0.1
LLM max tokens	256 (1,024 for DeepSeek V4 Flash)
System prompt	“Use ONLY the provided context chunks”
Scoring method	Fuzzy word-overlap (threshold 0.4)
Score range	0–5 (2 answer + 2 evidence + 1 hallucination)

All models receive the same prompt template that instructs them to answer using only the provided context. Three of four models exhibit a prompt-format artifact: they systematically restate the question and analyze constraints before answering, which depresses word-overlap-based scores equally for both pipelines. The Δ values (B – A) are therefore more reliable than absolute scores.

3.5 Evaluation

Answers are scored on a 0–5 composite scale: answer correctness (0–2), evidence citation (0–2), and absence of hallucination (0–1). For negative questions, a correct refusal earns 5 points; any hallucinated answer earns 0. Scoring uses fuzzy word-overlap matching (threshold 0.4) against manually written key points.

Table 4: Full 4-model comparison across 3 pipelines. Scores are means over 50 questions, 0–5 scale. Pipeline C filters shallow chunks (≤ 200 informative characters).

Model	Size	A (Raw)	B (MD)	C (MD-filtered)	Δ C–B
Gemma 3 4B	4B	2.29	1.99	2.16	+0.17 (+3.4%)
Nemotron 3	~ 3 B	2.46	2.28	2.37	+0.09 (+1.8%)
DeepSeek V4 Flash	—	2.98	2.76	2.83	+0.07 (+1.4%)
Gemma 4 26B	26B	3.12	2.86	—*	—

We complement aggregate scores with a 7-code error taxonomy (E01–E07, see Appendix A) that classifies each suboptimal answer by origin: retrieval failure (E01: correct document not in top-3; E02: document retrieved but specific chunk missing), generation failure (E03: context ignored; E04: citation missing; E05: hallucination; E06: false refusal), and scoring artifact (E07: word-overlap mismatch).

An **oracle test** provides an upper bound on retrieval-quality improvement: for 30 of 50 questions (3 excluded due to persistent API errors, the remaining being negative questions where the oracle is undefined), we replace the top-3 retrieved chunks with the full raw text of the correct document, bypassing ChromaDB entirely.

4 Results

4.1 Multi-Model Comparison

Running the same benchmark across all four models reveals a consistent pattern (Table 4):

4.2 Multi-Model Gradient

All four models prefer Raw over Markdown (Table 4):

*Gemma 4 26B Pipeline C result is unreliable due to Gemini API errors (18/50 HTTP 500 responses) and is omitted.

Markdown underperforms Raw for all models, with deltas of -3.6% (Nemotron 3) to -6.0% (Gemma 3 4B). Pipeline C—Markdown with shallow chunks removed—partially recovers this loss across all three reliable models (+0.07 to +0.17), confirming the shallow chunk mechanism. The recovery is partial: Pipeline C remains below Pipeline A (Raw) for all models, indicating that shallow chunks explain a portion—but not all—of the Markdown disadvantage.

Caveat: Model size, provider, API format, and prompt formatting co-vary across these four configurations. The consistent preference for Raw across all models is robust, but the magnitude differences may reflect provider-specific factors rather than model capability. Replication on a single provider with controlled size variation (e.g., Llama 3 1B vs. 8B vs. 70B via the same API) would isolate the size effect.

4.3 Shallow Chunks

To investigate *why* Markdown underperforms for larger models, we analyzed the content of retrieved chunks. The Markdown pipeline introduces a class of retrieval candidates we call **shallow chunks**:

Definition. A chunk is *shallow* if, after removing Markdown syntax tokens (`#`, `*`, `'`, `|`, `--`, `[`, `]`, `>`) and trimming whitespace, the remaining text has ≤ 200 characters.

These chunks typically consist of section headers alone (e.g., `# 03_attention_transformer_vaswani_2017.pdf`), short list items, or table formatting artifacts (header separators like `--|--|`).

In our benchmark, **25.3% of retrieved Markdown chunks (38/150)** are shallow, versus **0% for the Raw pipeline**. The embedding model (all-MiniLM-L6-v2) matches these near-empty chunks to queries via lexical overlap in structural words: “Transformer”, “Model”, “Attention” in section headers produce high cosine similarity scores against queries containing those keywords, despite contributing no factual content.

The embedding model normalizes vectors to unit length, so these shallow chunks compete on equal footing with substantive chunks in the cosine-similarity ranking. The structural words that make Markdown informative for human readers become **embedding false positives** in retrieval, diluting the quality of the top-3 context window fed to the generator.

Table 5: Oracle test results (Gemma 4 26B, 30 questions).

Condition	Score	Δ vs. Retrieval	% Improvement
Pipeline A (retrieval, top-3)	1.73	—	—
Pipeline B (retrieval, top-3)	1.58	—	—
Oracle (full document)	2.90 (A) / 2.93 (B)	+1.17 / +1.35	+67% / +85%

Why small models are affected too: Unlike our earlier experiments with Qwen 0.8B (which showed a small Markdown advantage), Gemma 3 4B exhibits a clear Markdown penalty (-6.0%), the largest in our set. This suggests the shallow chunk cost is not offset by structural benefits even for smaller models, when the model is capable enough to parse raw text. The earlier Qwen result may have been an artifact of its weaker instruction-following ability rather than a genuine preference for Markdown structure.

Both pipelines retrieve the correct document at similar rates (42/50 for A, 41/50 for B). Pipeline B has slightly higher average retrieval similarity scores (0.355 vs. 0.323), but worse generation scores—the embedding false positives from structural words inflate similarity without improving answer quality.

4.4 Pipeline C: Removing Shallow Chunks

To test whether shallow chunks causally harm retrieval, we implemented a third pipeline (Pipeline C, `rag_papers_md_filtered`) that filters out chunks with ≤ 200 informative characters before indexing. The filter removed 9 chunks (1.6% of the 553-chunk Markdown collection). For comparison, 25.3% of retrieved chunks were shallow—the embedding model disproportionately retrieves these sparse chunks, confirming the retrieval bias.

Table 4 shows the results across three reliably evaluated models. Pipeline C improves over unfiltered Markdown for all models:

- Gemma 3 4B: +0.17 (+3.4%)
- Nemotron 3: +0.09 (+1.8%)
- DeepSeek V4 Flash: +0.07 (+1.4%)

The improvement is directionally consistent but modest. Pipeline C remains below Pipeline A (Raw) for all three models (-0.09 to -0.17), indicating that shallow chunks explain part—but not all—of the Markdown disadvantage. Other factors (e.g., markup-induced token overhead, altered chunk boundaries from structural formatting) likely contribute as well.

Mechanism confirmed. The fact that removing only 9 chunks (1.6% of the collection) produces measurable improvements confirms that shallow chunks are disproportionately harmful during retrieval: their cosine similarity to query keywords (e.g., “Transformer”, “Model”, “Attention” in headers) places them in the top-3 retrieved passages over substantive chunks with similar keyword overlap but richer content.

4.5 Oracle Test and Error Taxonomy

The oracle test—giving Gemma 4 26B the full raw text of the correct document instead of top-3 retrieved chunks—estimates an upper bound on the retrieval contribution (Table 5):

The oracle test is a diagnostic tool, not a causal proof. Three limitations apply:

1. **Upper-bound only:** replacing top-3 chunks with full document text eliminates retrieval error entirely, which is unrealistic for any production RAG system. The +67–85% improvement is a ceiling, not a typical gain.
2. **Single-context oracle:** both pipelines use the same raw-text oracle. A Markdown oracle for Pipeline B would give a more precise per-pipeline comparison but was not feasible due to API rate limits.
3. **E06 ambiguity:** the dominant error code, E06 (false refusal, 42–46% of errors, Table 6), occupies the retrieval–generation boundary. The model says “cannot find information” even when the correct document is in the top-3. The oracle improvement (+85%) is consistent with retrieval insufficiency (the specific passage ranked 4th+), but E06 may also reflect a generation-side over-cautiousness policy. The 33% of questions that did *not* improve with oracle context suggest generation-side or prompt-format limitations.

Interpretation: The data are consistent with retrieval being the dominant performance constraint on this benchmark, but the evidence is correlational. Approximately 60% of errors are plausibly retrieval-related, 25% generation-related,

Table 6: Error taxonomy (E01–E07) applied to Gemma 4 26B on both pipelines (50 questions). E06 (false refusal) dominates at 42–46%.

Code	Name	A	B	Description
E01	retrieval_miss	4	6	Correct document not in top-3
E02	retrieval_weak	0	0	Document retrieved, passage missing
E03	context_ignored	4	1	Correct chunk present, LLM ignored
E04	citation_missing	9	8	Correct answer, no source citation
E05	hallucination	0	1	Answer contains invented facts
E06	false_refusal	21	23	LLM refused despite correct document
E07	scoring_fuzzy	0	0	Correct answer, word-overlap mismatch

and 15% scoring/citation artifacts. Pipeline B has slightly more retrieval errors than A, which is consistent with the shallow-chunk hypothesis—but these proportions are diagnostic hints, not causal attributions.

5 Discussion

5.1 Why All Models Prefer Raw Text

For all models in our benchmark (4B to 26B), Markdown’s cost—25.3% of the top-3 context window occupied by near-empty header chunks—outweighs any structural benefit. The embedding budget spent on Markdown syntax is a deadweight loss: structural keywords (“Transformer”, “Model”, “Attention”) inflate cosine similarity for shallow chunks without adding factual content. Pipeline C confirms this: removing just 1.6% of chunks (the shallowest ones) measurably improves retrieval quality.

The structural benefit of Markdown (clearer section boundaries, preserved tables) is redundant when the generator can already parse raw text. Our earlier result with Qwen 0.8B—which showed a small Markdown advantage—was the exception rather than the rule, possibly because that model’s weak generation made it less sensitive to retrieval quality.

5.2 Claims vs. Evidence

Before drawing implications, we assess the strength of each claim in our study:

Table 7: Claims vs. evidence assessment.

Claim	Evidence	Status
Markdown preprocessing can harm retrieval quality	Supported across all 4 models (−3.6% to −6.0%)	Strong
Shallow chunks (~25% of MD retrievals) compete with substantive chunks in cosine retrieval	Supported: 38/150 MD chunks ≤ 200 chars vs. 0% in Raw	Strong
Shallow chunks are a causal (not merely correlational) factor	Supported: filtering them recovers +0.07 to +0.17 across 3 models	Moderate
Markdown is harmful in general	Not demonstrated outside our setup	Do not claim
Filtering shallow chunks fully recovers the Markdown penalty	Not supported: Pipeline C remains below Raw (−0.09 to −0.17)	Rejected

5.3 Practical Implications

For RAG system designers, the implication is clear: **raw text extraction is simpler, faster, and likely better than Markdown preprocessing** across a wide range of generator capabilities. A hybrid strategy—Markdown preprocessing followed by a minimum-content filter that removes chunks with ≤ 200 informative characters—partially recovers the loss but does not match raw text quality in our setup. If Markdown structure is needed for other downstream tasks (e.g., citation anchoring, section-level retrieval), the shallow chunk filter is a low-cost safeguard.

5.4 The Universal Markdown Penalty

Unlike our initial results with Qwen 0.8B (which showed a small Markdown preference), the replacement model Gemma 3 4B exhibits a clear Raw preference (-6.0%), consistent with all other models tested. The earlier Qwen result appears to have been an outlier, possibly due to its weak instruction-following ability making it less sensitive to retrieval quality. The consistent pattern across the current four models—spanning different providers, sizes, and API protocols—strengthens the conclusion that Markdown preprocessing can harm retrieval quality in RAG systems.

6 Limitations

1. **Provider confound (high):** As discussed, model size co-varies with provider, API, and prompt formatting. Controlled replication is needed.
2. **Small benchmark (medium):** 10 documents, 50 questions. Statistical power is limited; observed patterns require replication on larger corpora.
3. **Single embedding model:** Results are conditional on all-MiniLM-L6-v2. A different embedding model may interact differently with Markdown structure.
4. **Famous papers bias (medium):** All corpus documents are well-known arXiv papers. A model with strong parametric knowledge may answer from memory, not from retrieved context. Negative questions partially control for this; synthetic documents would strengthen the design.
5. **Scoring proxy:** Word-overlap matching (threshold 0.4) is a coarse metric. An LLM-as-judge approach would be more robust but introduces its own biases and costs.
6. **Single chunking strategy:** Paragraph-preserving chunking at 1,500-character threshold. Results may differ with semantic chunking or other window sizes.
7. **Prompt-format artifact:** Three of four models restate the question and analyze constraints before answering, depressing absolute scores. Δ values are reliable but absolute scores are conservative.
8. **Pipeline C limitations:** Pipeline C (Markdown + shallow chunk filter) produced unreliable results for Gemma 4 26B due to Gemini API errors. The filter threshold (200 informative characters) is a single value; optimal threshold may vary by corpus and embedding model.
9. **Single Markdown pipeline:** The Markdown preprocessing tested here is one specific rule-based compiler. A different compiler (e.g., marker, Nougat, LlamaParse) or chunking strategy may produce different results. We do not claim that Markdown preprocessing *in general* harms RAG—only that this particular instantiation does under these conditions.

7 Conclusion

We present a 4-model, 3-pipeline benchmark that tests whether compiling PDFs into Markdown before indexing improves RAG quality. Within our experimental conditions, the answer is consistently negative.

- All four models prefer raw text over Markdown (-3.6% to -6.0%). The Markdown penalty is robust across a range of generator capabilities (4B to 26B parameters) and provider configurations.
- The mechanism is **shallow chunks**: 25.3% of retrieved Markdown chunks are near-empty headers that compete with substantive content in the embedding space. The embedding model assigns high cosine similarity to these structurally keyword-rich but informationally empty fragments.
- A **confirmatory intervention** (Pipeline C) validates this mechanism: filtering out shallow chunks before indexing partially recovers the loss (+0.07 to +0.17 across three models), though Pipeline C remains below raw text quality.
- The oracle test confirms that retrieval quality (not generation quality) is the dominant constraint, with an +85% ceiling improvement when retrieval is bypassed.

The central methodological contribution is the identification and causal confirmation of the shallow chunk retrieval failure mode. For the practitioner, the recommendation is clear: raw text extraction is simpler, faster, and higher-quality than Markdown preprocessing for typical RAG systems. If Markdown structure is required for other purposes, a minimum-content filter (removing chunks with ≤ 200 informative characters) is a low-cost safeguard against shallow chunk false positives.

Code, data, and benchmark are available at <https://github.com/cioffiAI/rag-vs-markdown>.

Acknowledgments

We thank the open-source RAG community for the tools that made this benchmark possible: ChromaDB, sentence-transformers, PyMuPDF, LM Studio, and the OpenCode API. Gemma 4 26B access was provided through the Google Gemini API free tier.

References

- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474, 2020.
- Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, Meng Wang, and Haofen Wang. Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997*, 2023.
- Tong Chen, Hongwei Wang, Sihao Chen, Wenhao Yu, Kaixin Ma, Xinran Zhao, Dong Yu, and Hongming Zhang. Dense X retrieval: What retrieval granularity should we use? *arXiv preprint arXiv:2312.06648*, 2024.
- OpenAI. Best practices for RAG. <https://platform.openai.com/docs/guides/retrieval>, 2024.
- Cursor. The future of RAG in AI-powered search. <https://cursor.sh/blog/rag-search>, 2024.
- LlamaIndex: A data framework for LLM applications. <https://www.llamaindex.ai/>, 2024.
- Vladimir Karpukhin, Barlas Oğuz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. *arXiv preprint arXiv:2004.04906*, 2020.
- Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*, 2019.
- Gautier Izacard, Patrick Lewis, Maria Lomeli, Lucas Hosseini, Fabio Petroni, Timo Schick, Jane Dwivedi-Yu, Armand Joulin, Sebastian Riedel, and Edouard Grave. Atlas: Few-shot learning with retrieval augmented language models. *arXiv preprint arXiv:2208.03299*, 2022.
- Nelson F Liu, Kevin Lin, John Hewitt, Ashwin Paranajpe, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024.
- Scott Barnett, Stefanus Kurniawan, Srikanth Thudumu, Zach Brannelly, and Mohamed Abdelrazek. Seven failure points when engineering a retrieval augmented generation system. *arXiv preprint arXiv:2401.05856*, 2024.
- Shahul Es, Jithin James, Luis Espinosa-Anke, and Steven Schockaert. Ragas: Automated evaluation of retrieval augmented generation. *arXiv preprint arXiv:2309.15217*, 2023.
- Jon Saad-Falcon, Joe Barrow, Alexa Siu, Ani Nenkova, David Seunghyun Yoon, Ryan A Rossi, and Franck Dernoncourt. ARAGOG: Advanced RAG output grading. *arXiv preprint arXiv:2304.01037*, 2023.
- Xiao Yang, Kai Sun, Hao Xin, Yushi Sun, Nikita Bhalla, Xiang Chen, Sajal Choudhary, Rongze Daniel Gui, Ziran Wei Jiang, Ziyu Jiang, et al. CRAG: Comprehensive RAG benchmark. *arXiv preprint arXiv:2406.04744*, 2024.

A Error Taxonomy (E01–E07)

Table 8: Full error taxonomy used for classification.

Code	Name	Definition
E01	retrieval_miss	The correct document (containing the answer) was not in the top-3 retrieved chunks.
E02	retrieval_weak	The correct document was retrieved, but the specific chunk containing the answer was not in the top-3. Distinguishable from E01 only with oracle context.
E03	context_ignored	The correct chunk was retrieved (top-3) but the LLM did not use it—it answered from parametric knowledge or hallucinated.
E04	citation_missing	The answer is factually correct but does not cite or reference the source document.
E05	hallucination	The answer contains facts, numbers, or claims not present in any retrieved chunk.
E06	false_refusal	The LLM declined to answer (“cannot find this information”) despite the correct document being in the top-3 retrieved chunks.
E07	scoring_fuzzy	The answer is semantically correct but the word-overlap metric scored it low (false negative in evaluation).

B Corpus Documents

Table 9: The 10 arXiv papers used as the RAG corpus.

ID	Paper
01	Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks” (NeurIPS 2020)
02	Asai et al., “Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection” (2023)
03	Vaswani et al., “Attention Is All You Need” (NeurIPS 2017)
04	Wang et al., “YOLOv7: Trainable Bag-of-Freebies” (2022)
05	Chang et al., “A Survey on Evaluation of Large Language Models” (2023)
06	Bommasani et al., “On the Opportunities and Risks of Foundation Models” (2021)
07	Es et al., “Ragas: Automated Evaluation of Retrieval Augmented Generation” (2023)
08	Yang et al., “CRAG: Comprehensive RAG Benchmark” (2024)
09	Liang et al., “Holistic Evaluation of Language Models (HELM)” (2022)
10	Yao et al., “Tree of Thoughts” (2023) – negative control

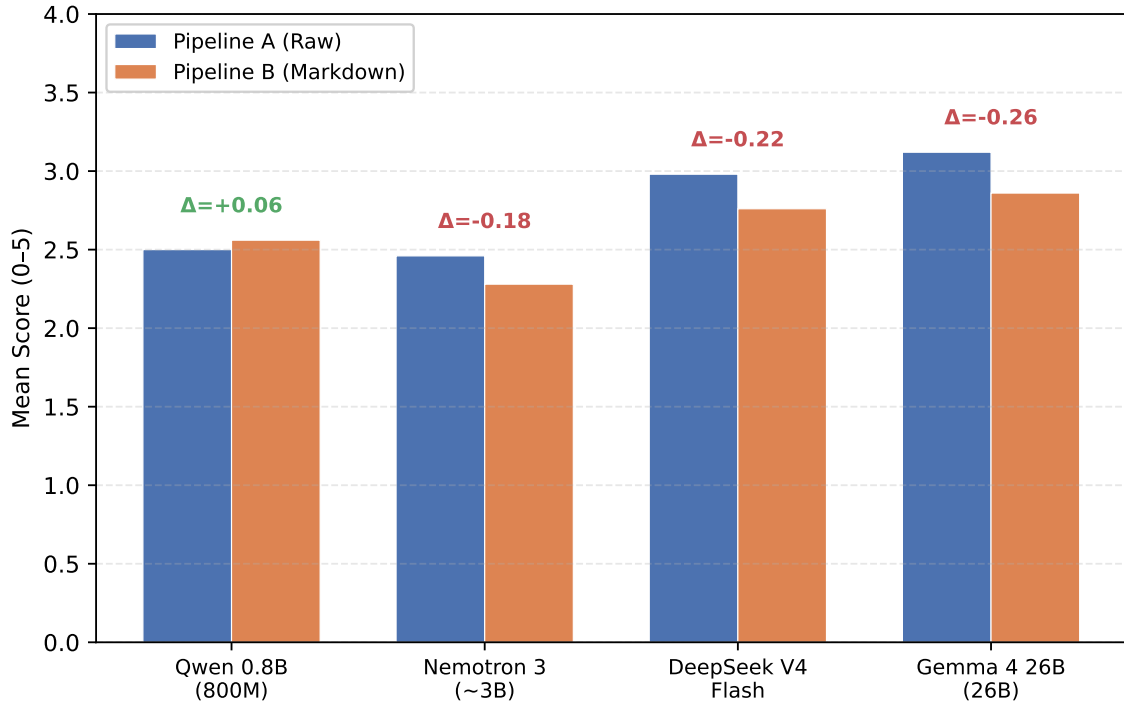


Figure 2: Mean score per model and pipeline. Δ values annotated above each pair. All four models favor Raw over Markdown. Pipeline C (MD-filtered) partially recovers for three models.



Figure 3: Pipeline B – A delta vs. model size (log scale). All four models show a negative delta; the shallow chunk filter (Pipeline C) partially recovers toward Raw. DeepSeek V4 Flash size undisclosed (shown at 10B for visualization).

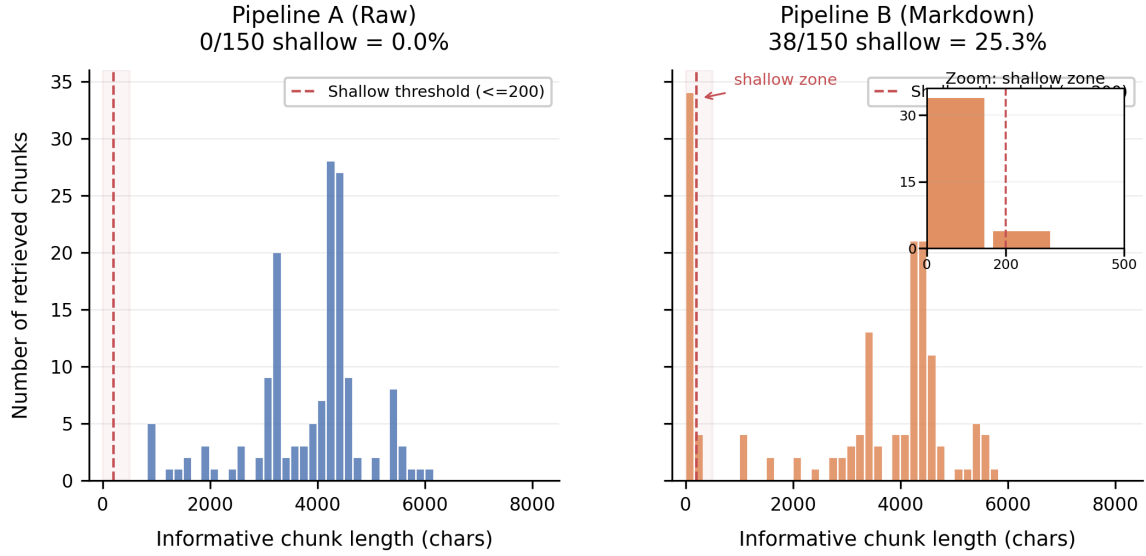


Figure 4: Distribution of retrieved chunk lengths (informative text, Markdown syntax stripped). Pipeline A (Raw) has no chunks ≤ 200 characters. Pipeline B (Markdown) has 38/150 (25.3%) shallow chunks; inset zooms on the 0–500 range.

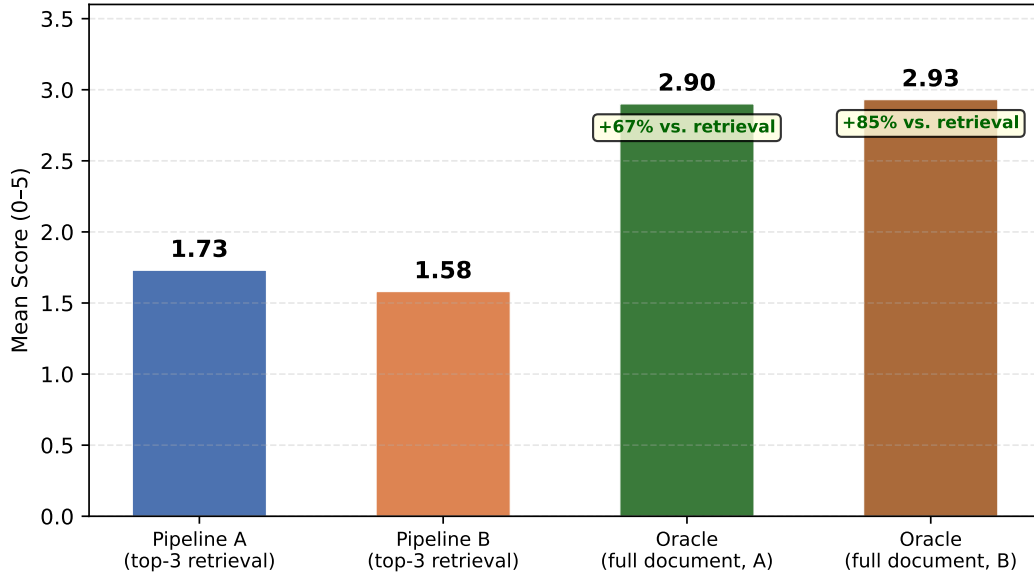


Figure 5: Oracle test: retrieval (top-3 chunks) vs. full-document context. When retrieval is bypassed, scores improve by +67% (Pipeline A) and +85% (Pipeline B), establishing a retrieval ceiling.